Date: Day:

→ uses memory!

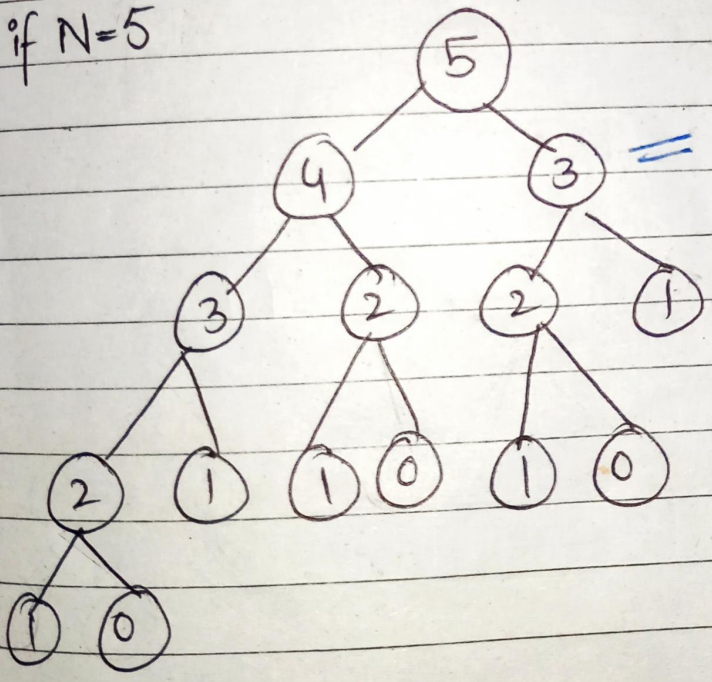
# Dynamic Programming

- Overlapping sub-problems → recalculating the same subproblem
- Optimal sub-structure

```
fib (N) {
    if (N == 0 || N == -1)
        return N;
    return (fib(N-1) + (fib(N-2));
}
```

Brute force

if N=5




= recalculate

dp: (calculate & store it).

① fib[1 .... N]

Fib[1] = 1

Fib[2] = 1

// Allocate memory

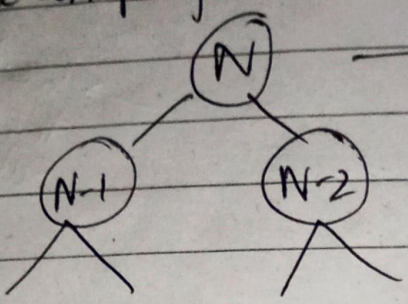
Fib i = 3 to N

Fib[i] = Fib[i-1] + Fib[i-2]

return fib[N];

Date: Day:

Time Comp for recursive :-




— 1

— 2

— 4

— — — — — $2^N$

$1 + 2 + 4 + \cdots\cdots + 2^N$

$= O(2^N)$

# Maximum SubArray Sum using DP → O(N)

| Arr | -1 | 3 | 4 | -5 | 9 | -2 |
|---|---|---|---|---|---|---|
| | 1 | 2 | 3 | 4 | 5 | 6 |

| Res | -1 | 3 | 7 | 2 | 11 | 09 |
|---|---|---|---|---|---|---|
| | 1 | 2 | 3 | 4 | 5 | 6 |

max subarray ending at index i

↳ ~~max(A[i], A[i]+f[i-1])~~

$Res[i] = \max(A[i], A[i] + Res[i-1])$

Max. Subarray ending at index #i = max ( elem at index #i

elem at index i + max SubArr ending at "i-1" )

# Lecture # 13

Date: Oct 2nd, 25 Day: Thursday

## Optimization problem

→ Results are either minimized or maximized.
→ Solved through DP or Greedy method.

## Maximum Subarray sum (code)

A (input Array)

| 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|
| -1 | 3 | 4 | -5 | 9 | -2 |

R (dp-Array) : max subarray sum ending at index 'i'

| -1 | 3 | 7 | 2 | 11 | 9 |
|---|---|---|---|---|---|

~~All~~ subarrays ending at index # 05

$$\max \begin{pmatrix} -1+3+4-5+9 \\ 3+4-5+9 \\ 4-5+9 \\ -5+9 \\ 9 \end{pmatrix}$$

→ T(N) = O(N)

```
MaxSubArray ( A[ ], N) {

    R [0 ---- N]
    R[0] = A[0]

    int overAllMax = R[0]
for  i = 1  to  N {
        R[i] = max( A[i], A[i] + R[i-1])
    if ( overAllmax < R[i])
            overAllMax = R[i]
                }
    setuon overAllMax;
                }
```

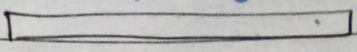
→ overAllMax = Max(R[i], overAllmax)

Date: Day:

# Rod Cutting: (Profit maximization).

→ A rod of length 'N'

→ Cut in diff. sizes..




Possible Solutions:

5cm = 1+1+1+1+1 (5 slices)

= 1+1+1+2 (4 ")

= 1+2+2 (3 ")

= 2+3 (2 ")

= 1+4 (2)

= 5 (1)

Prices:

| 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|
| $3 | $4 | $5 | $6 | $7 |

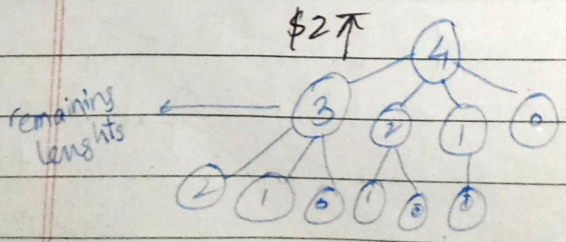
↳ 5 * 3 = $15

Example:

| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|---|---|---|---|---|---|---|---|---|---|
| 1 | 5 | 8 | 9 | 10 | 17 | 17 | 20 | 24 | 30 |

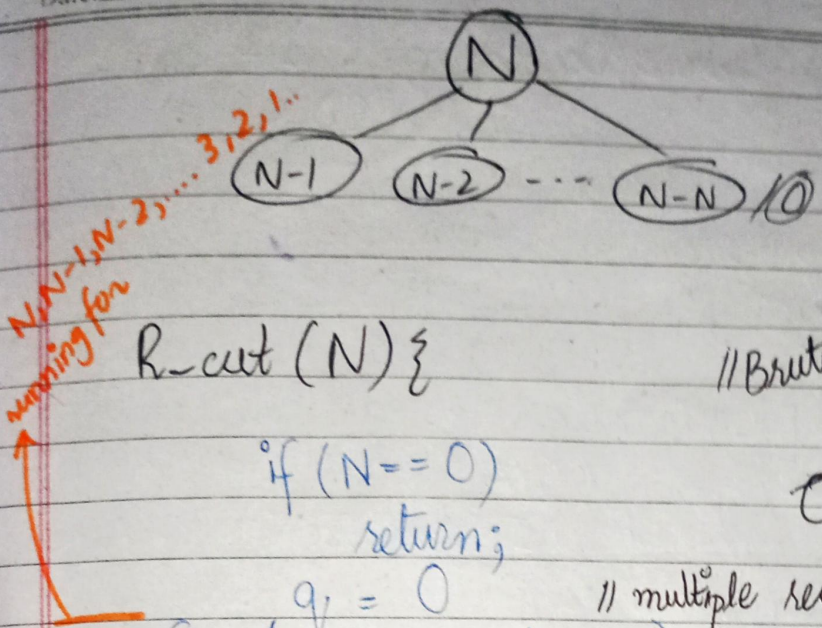
| Price | 2 | 5 | 10 | 11 |
|---|---|---|---|---|

N = 4




max ( 1m + 3m, 2m + 2m, 3m + 1m, 4m ) 8

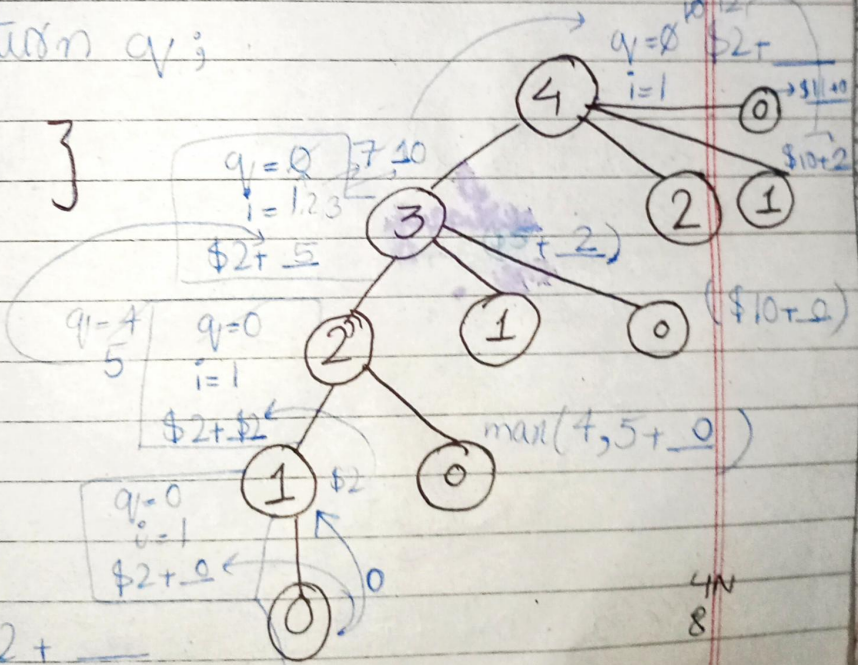
Date: Day:




```
R_cut(N){
    if(N==0)
        return;
    q = 0                    // multiple results.
    for( i = 1 to N )        //control the slice length
        q = max(q, R_cut(N-i) + Pr[i])
    return q;
}
```

// Brute Force Approach?

$O(2^{N-1})$




max( $2+ ___
$5+ ___
$10+ ___
$11+ ___ )

4N
8

S → aB bA
bbAA
bb

```
Rod-cut (N){
    if (N == 0)
        return 0;
        - - - -
}
```

// Bottom up Iterative code :-

```
Rod-cut (N){                                  —— O(N^2)

    R [0 - - - - N]
    R[0] = 0;
for ( j = 1 to N) {          → control the length of rod.
        q = 0
    for( i = 1 to j) {       → slicing
    q = max (q, R[j-i] + Pr[i])
                }
    R[j] = q

    return R[N]
}
```

# Lecture # 14

Date: 7 Oct 2025 Day: Tuesday

## Longest Common Subsequence (LCS)

$a_1, a_2, a_3, \ldots\ldots, a_n$

subsequence : $a_2, a_1, a_9, \ldots$

stone : sub-seq : (s,t,o,n,e), (s,t,o), (o,n,e) (s,o,e) ....

* stone, softec

(s,o,e), (s,o), (s), (o), (e), (s,t)

return = 3

Brute Force Solution:- (recursive) Top down App

```
LCS (N, M) {
  if (N == 0 || M == 0)
        return 0;
```

N to 1 ; M to 1

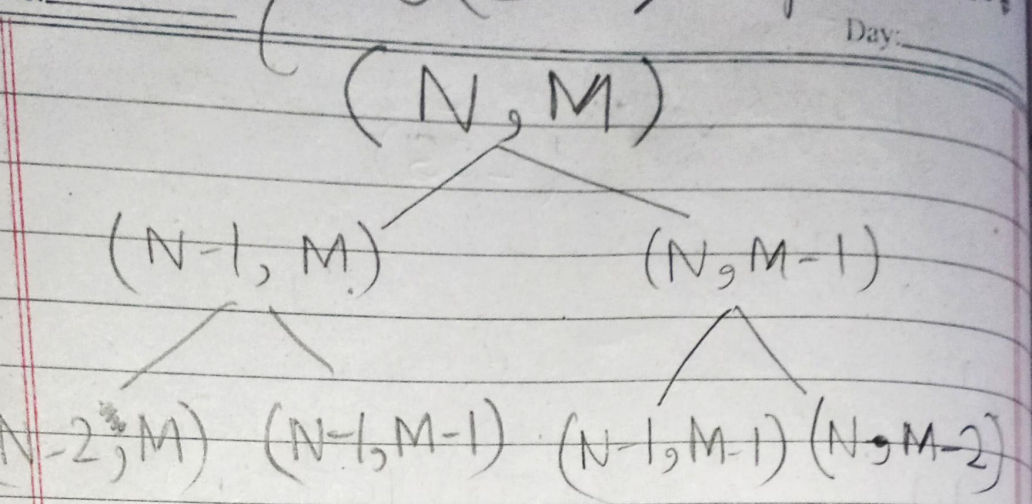
```
  if (A[N] == B[M])
     return 1 + LCS (N-1, M-1)

  else
  return max (LCS (N-1, M), LCS (N, M-1))

}
```

ate: Day:

$O(2^{N+M})$ exponential?




Dynamic Programming:

LCS(N, M) → $O(N*M)$

Res[0 --- N][0 --- M]

```
for(i = 0 to N)
    Res[i][0] = 0
for(j = 0 to M)
    Res[0][j] = 0
```

filling the matrix — Base case.

```
for i = 1 to N
    for j = 1 to M
    {
        if (A[i] == B[j])
            R[i][j] = 1 + Res[i-1][j-1]
        else
            Res[i][j] = max(Res[i-1][j], Res[i][j-1])
    }
    }
    return Res[N][M];
```

# Lecture # 15

Date: Oct 9th, 25 Day: Thurs

## (Binary knapsack)

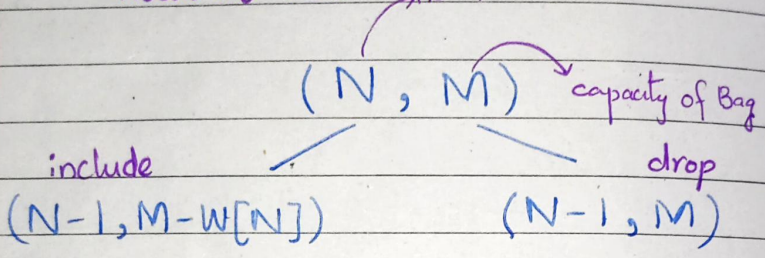
M = 10 kg (Capacity of knapsack)

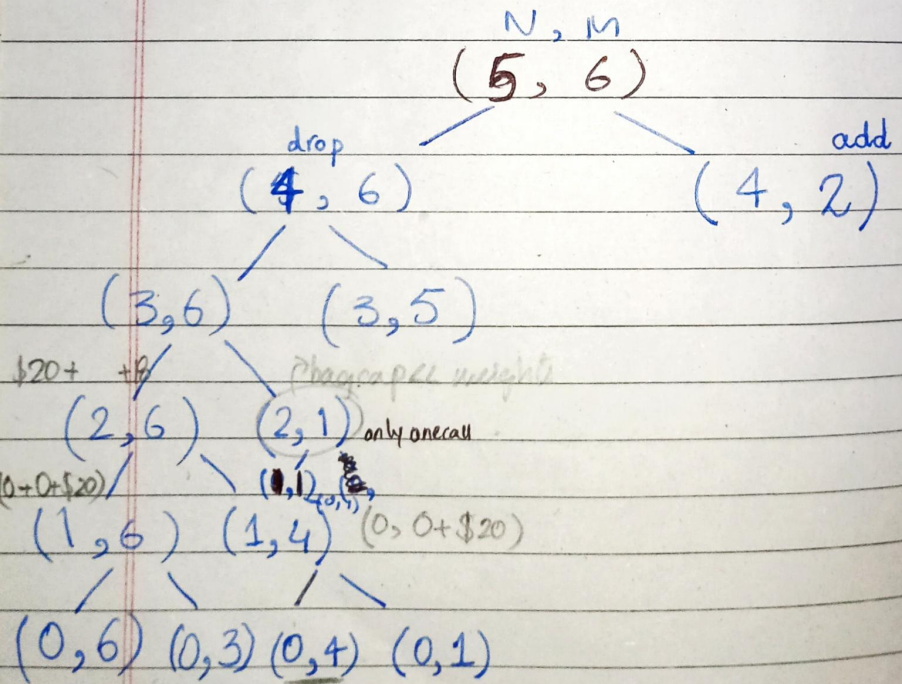
weights: W[1 --- N] = { 3, 2, 1, 6, 4, 5 }

V[1 --- N] = { $15, $10, $16, $12, $13, $18 }

Maximize the profit

Recursive Tree:



Date: Day:

## Brute Force Code:

```
KSP(N, M){
    if(N == 0 || M == 0)
        return 0;
    if(M < W[N])          // bagcap < weight of item
        return KSP(N-1, M)   // dropped that item
    else
    return max(KSP(N-1, M), KSP(N-1, M-W[N]) + V[N]);
```

Time Complexity = $T(2^N * 2^M)$